HOLY ANGEL UNIVERSITY | SCHOOL OF COMPUTING

6DANCS - Data Analytics for Computer Science
2nd Semester (SY 2025-2026)

Final Project Documentation

This document describes how students can apply data analytics and machine learning concepts to analyze a real-world dataset. The project involves data preparation, exploratory analysis, model training, and interpretation of results. Students are expected to demonstrate both technical skills and analytical reasoning.

Section: CS 303	Date Submitted: MARCH 31, 2026
Group Members: ALVARADO, RAFAEL ALLAN DIZON, GHAYZELLE MALIG, LOUIS GABRIEL MANANGU, REDENELLE MAURICE	
Project Title: CUSTOMER SUPPORT ASSISTANT (NLP + RL)	

Project Description

This project builds a Customer Support Assistant for the e-commerce domain, combining Natural Language Processing (NLP) and Reinforcement Learning (RL) to automatically classify customer intent and select appropriate responses. The dataset used is the Bitext Customer Support LLM Chatbot Training Dataset, a publicly available, synthetically generated benchmark sourced from HuggingFace (Bitext, 2023). It contains 27,000 labeled customer support utterances across 27 e-commerce intent categories such as tracking orders, requesting refunds, cancelling orders, and handling payment issues.

The system uses DistilBERT-base-uncased as the core NLP model for intent classification, an epsilon-greedy contextual bandit (LinUCB) as the RL component for response selection, and a TextCNN as a supporting CNN experiment and comparison baseline. The dataset was preprocessed, split, and used to train and evaluate all three components. This project serves as an academic prototype demonstrating end-to-end AI system design for customer support automation.

Project Objectives

- To design a DistilBERT-based intent classification model capable of categorizing e-commerce customer support messages into 27 intent classes, achieving a macro-F1 score of at least 0.85 on the held-out test set.
- To implement a TextCNN as a supporting CNN experiment for intent classification using GloVe word embeddings, with ablation studies across embedding modes (static vs. fine-tuned) and filter size configurations ([2,3,4] and [3,4,5]), to serve as a comparative DL baseline against DistilBERT.
- To evaluate an epsilon-greedy contextual bandit (LinUCB) for automated response selection, ensuring that the RL agent achieves a task success rate of at least 70% and demonstrates a measurable improvement in cumulative reward over a random response baseline.

Data Preparation

The dataset was sourced from HuggingFace as the Bitext Customer Support LLM Chatbot Training Dataset (Bitext, 2023), available under a Creative Commons Attribution 4.0 International (CC BY 4.0) license. It is synthetically generated and contains no personally identifiable information (PII). The raw file contained the columns instruction and intent, which were renamed to text and intent respectively during loading. All preprocessing steps were implemented in data/get_data.py and are described below.

The following preprocessing steps were applied in sequence:

- **Column selection and renaming**: Only the instruction and intent columns were retained from the raw CSV. The instruction column was renamed to text and the intent column was kept as-is.
- **Null removal**: Rows with any null values were dropped using dropna().
- **Text normalization**: All text was converted to lowercase and stripped of leading/trailing whitespace. Extra internal whitespace was collapsed using regex (\s+). Only alphanumeric characters and select punctuation (?, !) were retained; all other special characters were removed.
- **Intent normalization:** Intent labels were cast to string, converted to lowercase, and stripped of whitespace to ensure consistency across labels.

- **Duplicate removal**: Exact duplicate rows based on the (text, intent) pair were removed after cleaning.
- **Length filtering**: A word count column was computed for each utterance. Rows with zero words (empty text) were removed. Rows exceeding 128 words were also removed, though in practice no utterances exceeded 16 words.
- **Rare intent removal**: Intents with only a single sample were removed to ensure stratified splitting was possible, though all 27 intents in this dataset had sufficient samples.
- **Train/Validation/Test split**: An 80/10/10 stratified split was applied using train_test_split() from scikit-learn with random_state=42. The stratify parameter ensured proportional representation of all 27 intent classes across splits.

The final split sizes after preprocessing were as follows:

Split	Samples	Percentage
Train	19,348	80%
Validation	2,418	10%
Test	2,419	10%
Total	**24,185**	**100%**

Descriptive Analysis

Dataset Overview

The cleaned dataset contains **24,185 utterances** across **27 e-commerce intent categories**. Each record consists of a customer support message (text) and its corresponding intent label. The dataset is synthetically generated, meaning all utterances were programmatically created to simulate real customer support language across a range of e-commerce scenarios.

Utterance Length Statistics

The word count per utterance was computed and analyzed as a key descriptive feature:

Statistic	Value
Mean utterance length	8.86 words
Median utterance length	9 words
Mode utterance length	9 words
Standard deviation	2.56 words
Minimum length	1 word
Maximum length	16 words

The mean, median, and mode are all tightly clustered around 8–9 words, indicating a highly concentrated and consistent utterance length distribution with low variability (std = 2.56). This is consistent with the synthetic nature of the dataset, where utterances were generated to simulate short, direct customer queries. The narrow range (1–16 words) also confirms that no excessively long texts are present, making this dataset well-suited for short-context NLP models such as DistilBERT and TextCNN.

Intent Distribution

The 27 intent classes and their sample counts in the cleaned dataset are as follows:

Intent	Count	Intent	Count
complaint	996	check_cancellation_fee	939
place_order	992	check_invoice	913
review	992	get_refund	907
check_payment_methods	991	delete_account	897
payment_issue	991	create_account	874
delivery_period	990	get_invoice	874
newsletter_subscription	990	change_order	854
set_up_shipping_address	989	switch_account	822
recover_password	987	track_order	779
contact_customer_service	983	edit_account	765
registration_problems	979	track_refund	687
contact_human_agent	977	delivery_options	655
check_refund_policy	974	cancel_order	428
change_shipping_address	960		

The dataset is generally well-balanced across most of the 27 intents, with the majority of classes containing between 850 and 996 samples and an average of approximately 895.74 samples per intent. However, a notable class imbalance is observed in cancel_order, which has only 428 samples, which is less than half the count of the most frequent class (complaint, 996). Similarly, delivery_options (655) and track_refund (687) are also underrepresented relative to the average. This imbalance was flagged as a potential fairness concern, as models trained on this data may exhibit lower recall on these underrepresented classes.

Patterns and Anomalies

- **Trend:** The dataset exhibits a clear cluster of high-frequency intents (complaint, place_order, review, check_payment_methods, payment_issue) all sitting near the 990–996 sample range, suggesting these were intentionally over-sampled to reflect the most common real-world customer concerns in e-commerce.

- **Anomaly:** The cancel_order intent has 428 samples, a count that stands out as significantly lower than all other classes. No other intent drops below 600. This is likely a deliberate design choice in the synthetic dataset to reflect the relatively lower frequency of cancellation requests compared to tracking or payment queries in real e-commerce platforms. However, it introduces a modeling risk where the classifier may underperform on this class.

- **Pattern:** All utterance lengths fall between 1 and 16 words with low variance, indicating that the synthetic generation process was constrained to produce short, direct messages. This is consistent with real-world customer support messages, which tend to be brief and task-oriented.

Training Process

Non-DL Baseline: TF-IDF + Logistic Regression As the non-deep learning baseline, a Term Frequency-Inverse Document Frequency (TF-IDF) vectorizer was applied to transform raw text utterances into numerical feature vectors. TF-IDF works by assigning higher weights to words that appear frequently in a specific document but rarely across the entire corpus, making it effective at capturing discriminative vocabulary for each intent class. The vectorizer was configured with a bigram range (ngram_range=(1,2)), meaning it captures both individual words and two-word phrase combinations as features, and was limited to a maximum vocabulary of 10,000 features to control dimensionality. The resulting TF-IDF vectors were then passed to a Logistic Regression classifier configured with a regularization strength of C=5.0, a maximum of 1,000 solver iterations, and a fixed random seed of 42 for reproducibility. Logistic Regression was chosen as the non-DL baseline because it is interpretable, computationally inexpensive, and well-established as a strong text classification baseline in NLP literature. Both the vectorizer and classifier were fitted exclusively on the training split, and the fitted vectorizer was applied to transform the test split without refitting, ensuring no data leakage.

```python
section("Baseline — TF-IDF + Logistic Regression (non-DL)")

tfidf      = TfidfVectorizer(max_features=10000, ngram_range=(1, 2))
X_train_tf = tfidf.fit_transform(train_df["text"])
X_test_tf  = tfidf.transform(test_texts)

lr = LogisticRegression(max_iter=1000, random_state=42, C=5.0)
lr.fit(X_train_tf, train_df["intent"])
lr_preds = lr.predict(X_test_tf)

lr_acc = accuracy_score(test_labels_str, lr_preds)
lr_f1  = f1_score(test_labels_str, lr_preds, average="macro")

print(f"  Accuracy  : {lr_acc:.4f}")
print(f"  Macro-F1  : {lr_f1:.4f}")
print(f"\n{classification_report(test_labels_str, lr_preds, target_names=label_names)}")
```

```
============================================================
  Baseline — TF-IDF + Logistic Regression (non-DL)
============================================================
  Accuracy  : 0.9946
  Macro-F1  : 0.9937

                          precision    recall  f1-score   support

            cancel_order       1.00      0.91      0.95        43
            change_order       0.95      0.99      0.97        85
    change_shipping_address    0.99      1.00      0.99        96
    check_cancellation_fee     1.00      1.00      1.00        94
           check_invoice       0.98      1.00      0.99        92
    check_payment_methods      1.00      1.00      1.00        99
      check_refund_policy      1.00      1.00      1.00        98
               complaint       1.00      1.00      1.00       100
  contact_customer_service     1.00      0.99      0.99        99
      contact_human_agent      1.00      1.00      1.00        98
           create_account      0.99      0.99      0.99        87
           delete_account      0.99      1.00      0.99        89
         delivery_options      1.00      1.00      1.00        65
         delivery_period       1.00      1.00      1.00        99
            edit_account       0.99      1.00      0.99        76
              get_invoice      1.00      0.98      0.99        87
              get_refund       1.00      1.00      1.00        91
  newsletter_subscription      1.00      1.00      1.00        99
         payment_issue         1.00      0.98      0.99        99
            place_order        0.99      1.00      1.00       100
        recover_password       0.99      1.00      0.99        99
  registration_problems        0.99      0.99      0.99        98
                review         1.00      1.00      1.00        99
  set_up_shipping_address      1.00      1.00      1.00        99
          switch_account       1.00      1.00      1.00        82
            track_order        1.00      0.99      0.99        78
          track_refund         1.00      1.00      1.00        68

                accuracy                           0.99      2419
               macro avg       0.99      0.99      0.99      2419
            weighted avg       0.99      0.99      0.99      2419
```

DL Baseline: DistilBERT (NLP Core Model)

The core NLP model is DistilBERT-base-uncased, a distilled version of BERT. DistilBERT retains approximately 97% of BERT's language understanding capability while being 40% smaller and 60% faster to run, making it well-suited for academic prototypes with runtime constraints. Unlike TF-IDF which treats words as isolated features, DistilBERT produces deep bidirectional contextual representations meaning the embedding of each token is influenced by all other tokens in the sentence, allowing the model to capture nuanced semantic relationships. The model was fine-tuned on the project's training split using the HuggingFace Transformers Trainer API. Fine-tuning involves taking the pre-trained DistilBERT weights and continuing to train them on the task-specific dataset, updating the model parameters to adapt the general language representations toward the intent classification objective. The training configuration was as follows: AdamW optimizer with a learning rate of 2e-5 and weight decay of 0.01 to prevent overfitting, a per-device batch size of 16, a maximum of 5 training epochs, and early stopping with a patience of 2 epochs monitoring validation macro-F1 to halt training when performance stopped improving. Input utterances were tokenized using the DistilBERT WordPiece tokenizer with a maximum sequence length of 128 tokens, with padding and truncation applied as needed. At the end of each epoch, the model was evaluated on the validation split, and the checkpoint with the highest validation macro-F1 was automatically saved. The best checkpoint was loaded at the end of training and saved to experiments/results/fine_tuned_model/ for inference. Training metrics including loss, accuracy, and macro-F1 were logged per epoch via TensorBoard.

```python
training_args = TrainingArguments(
    output_dir="./results",
    eval_strategy="epoch",      # evaluate at the end of each epoch
    save_strategy="epoch",            # save checkpoint each epoch
    learning_rate=2e-5,
    per_device_train_batch_size=16,
    num_train_epochs=5,
    weight_decay=0.01,
    report_to="tensorboard",
    logging_steps=10,
    load_best_model_at_end=True,
    metric_for_best_model="macro_f1"
)

trainer = Trainer(
    model=model,
    args=training_args,
    train_dataset=tokenized_dataset["train"],
    eval_dataset=tokenized_dataset["validation"],
    compute_metrics=compute_metrics,
    callbacks=[EarlyStoppingCallback(early_stopping_patience=2)]
)
# Start training
trainer.train()

# Get predictions
predictions = trainer.predict(tokenized_dataset['test'])
predicted_labels = np.argmax(predictions.predictions, axis=1)
true_labels = tokenized_dataset['test']['labels']
```

```
============================================================
  DistilBERT — NLP + ML Model Evaluation
============================================================
Loading weights: 100%|██████████| 104/104 [00:00<00:00, 6877.64it/s]
Loading weights: 100%|██████████| 100/100 [00:00<00:00, 5503.25it/s]
DistilBertModel LOAD REPORT from: c:\Users\rafae\customer-support-assistant\project-root\src\..\experiments/results/fine_tuned_model
Key                | Status     |  |
-------------------+------------+--+
classifier.bias    | UNEXPECTED |  |
pre_classifier.bias  | UNEXPECTED |  |
pre_classifier.weight | UNEXPECTED |  |
classifier.weight  | UNEXPECTED |  |

Notes:
- UNEXPECTED    :can be ignored when loading from different task/architecture; not ok if you expect identical arch.
  Running inference on test set...
  Accuracy          : 0.9971
  Macro-F1          : 0.9967
  Low-conf (<0.70)  : 3 flagged

                          precision    recall  f1-score   support

            cancel_order       1.00      0.95      0.98        43
            change_order       0.97      1.00      0.98        85
    change_shipping_address    0.99      1.00      0.99        96
    check_cancellation_fee     1.00      1.00      1.00        94
           check_invoice       0.99      1.00      0.99        92
    check_payment_methods      1.00      0.99      0.99        99
      check_refund_policy      1.00      1.00      1.00        98
               complaint       1.00      1.00      1.00       100
  contact_customer_service     1.00      1.00      1.00        99
      contact_human_agent      1.00      1.00      1.00        98
           create_account      0.99      1.00      0.99        87
           delete_account      1.00      1.00      1.00        89
         delivery_options      1.00      1.00      1.00        65
         delivery_period       1.00      1.00      1.00        99
            edit_account       1.00      1.00      1.00        76
              get_invoice      1.00      0.99      0.99        87
              get_refund       1.00      1.00      1.00        91
  newsletter_subscription      1.00      1.00      1.00        99
         payment_issue         0.99      1.00      0.99        99
            place_order        1.00      0.99      0.99       100
        recover_password       1.00      1.00      1.00        99
  registration_problems        1.00      0.99      0.99        98
                review         1.00      1.00      1.00        99
  set_up_shipping_address      1.00      0.99      0.99        99
          switch_account       1.00      1.00      1.00        82
            track_order        1.00      1.00      1.00        78
          track_refund         1.00      1.00      1.00        68

                accuracy                           1.00      2419
               macro avg       1.00      1.00      1.00      2419
            weighted avg       1.00      1.00      1.00      2419

------------------------------------------------------------
  Slice Analysis (by utterance length):
    Short (<4 words) n=1076  Macro-F1: 0.9946
    Long  (≥8 words) n=1343  Macro-F1: 0.9994
------------------------------------------------------------
  Failure Cases (first 5 misclassified):
    [1] "cance order order number"
        True: cancel_order  |  Pred: change_order  |  Conf: 1.00
    [2] "shipping address issue"
        True: set_up_shipping_address  |  Pred: change_shipping_address  |  Conf: 0.89
    [3] "i lost the bill from person name send it to me"
        True: get_invoice  |  Pred: check_invoice  |  Conf: 0.97
    [4] "want help making a fucking order"
        True: place_order  |  Pred: change_order  |  Conf: 1.00
    [5] "help creating account"
        True: registration_problems  |  Pred: create_account  |  Conf: 0.75
```

CNN Component: TextCNN Ablation Study

The CNN component follows the TextCNN architecture, which applies parallel convolutional filters of varying sizes directly over word embedding matrices to capture local phrase-level patterns of different lengths. Unlike recurrent models that process sequences step

by step, convolutional filters slide across the embedding matrix simultaneously, making TextCNN computationally efficient. Word representations were initialized using pre-trained GloVe embeddings trained on 6 billion tokens with a vocabulary of 400,000 words and a dimensionality of 300. Words not found in the GloVe vocabulary were initialized as zero vectors. To satisfy the ablation requirement, four configurations were trained and compared. The first two ablations varied the embedding mode: in the static configuration, GloVe embedding weights were frozen during training so the model only learns the convolutional and classification layers; in the fine-tuned configuration, the embedding weights were also updated during backpropagation, allowing the model to adapt word representations to the specific vocabulary of the dataset. The second two ablations varied the convolutional filter sizes: configurations using filter sizes [2,3,4] capture bigram, trigram, and 4-gram patterns, while configurations using [3,4,5] capture trigram, 4-gram, and 5-gram patterns, with the latter designed to capture slightly longer local phrases. All four configurations used 50 filters per size, a dropout rate of 0.5 applied before the final classification layer, Adam optimizer with a learning rate of 0.001, a batch size of 16, a maximum of 50 training epochs, and early stopping with a patience of 2 epochs based on validation macro-F1. The best model checkpoint across all configurations was saved to cnn_snapshot/ using PyTorch's torch.save(). The best performing configuration was static_3_4_5, which achieved a macro-F1 of 0.9859 on the test set.

```python
# ----------------------------
# Text-CNN
# ----------------------------
class TextCNN(nn.Module):
    def __init__(self, embed_dim, num_classes, filter_sizes, num_filters=50, fine_tune=False):
        super().__init__()
        self.fine_tune = fine_tune
        self.convs = nn.ModuleList([
            nn.Conv2d(1, num_filters, (fs, embed_dim)) for fs in filter_sizes
        ])
        self.dropout = nn.Dropout(0.5)
        self.fc = nn.Linear(num_filters * len(filter_sizes), num_classes)

    def forward(self, x):
        if not self.fine_tune:
            x = x.detach()
        x = x.unsqueeze(1)
        convs = [torch.relu(conv(x)).squeeze(3) for conv in self.convs]
        pools = [torch.max(c, dim=2)[0] for c in convs]
        out = torch.cat(pools, dim=1)
        out = self.dropout(out)
        return self.fc(out)

EPOCHS = 50
PATIENCE = 2
all_metrics = []
best_model_path = None

# ----------------------------
# Training
# ----------------------------
for config in ablation_configs:
    print(f"\n=== Training {config['name']} ===")
    model = TextCNN(
        EMBEDDING_DIM,
        len(le.classes_),
        filter_sizes=config["filter_sizes"],
        fine_tune=config["fine_tune"]
    )

    optimizer = torch.optim.Adam(model.parameters(), lr=0.001)
    criterion = nn.CrossEntropyLoss()

    best_val_f1 = 0.0
    wait = 0
```

```
============================================================
  TextCNN — CNN Ablation Evaluation
============================================================
  Loading GloVe...
  GloVe loaded: 400000 words
  static_2_3_4            — Accuracy: 0.9773  Macro-F1: 0.9765
  fine_tuned_2_3_4        — Accuracy: 0.9814  Macro-F1: 0.9812
  static_3_4_5            — Accuracy: 0.9859  Macro-F1: 0.9859
  fine_tuned_3_4_5        — Accuracy: 0.9835  Macro-F1: 0.9829

  Best CNN: static_3_4_5 (Macro-F1: 0.9859)

                          precision    recall  f1-score   support

            cancel_order       0.98      0.98      0.98        43
            change_order       0.97      0.99      0.98        85
    change_shipping_address    0.99      0.99      0.99        96
    check_cancellation_fee     1.00      1.00      1.00        94
            check_invoice      0.96      1.00      0.98        92
    check_payment_methods      1.00      0.98      0.99        99
    check_refund_policy        0.99      1.00      0.99        98
              complaint        1.00      1.00      1.00       100
    contact_customer_service   1.00      0.98      0.99        99
      contact_human_agent      1.00      1.00      1.00        98
          create_account       0.97      1.00      0.98        87
          delete_account       1.00      0.97      0.98        89
        delivery_options       1.00      0.98      0.99        65
        delivery_period        0.99      1.00      0.99        99
           edit_account        0.99      0.99      0.99        76
           get_invoice         1.00      0.95      0.98        87
            get_refund         0.98      0.99      0.98        91
    newsletter_subscription    0.98      0.99      0.98        99
          payment_issue        0.91      0.99      0.95        99
            place_order        0.98      0.99      0.99       100
        recover_password       1.00      1.00      1.00        99
    registration_problems      1.00      0.89      0.94        98
                review         0.99      0.99      0.99        99
    set_up_shipping_address    0.99      1.00      0.99        99
          switch_account       0.99      1.00      0.99        82
           track_order         1.00      0.99      0.99        78
          track_refund         1.00      0.99      0.99        68

               accuracy                             0.99      2419
              macro avg        0.99      0.99      0.99      2419
           weighted avg        0.99      0.99      0.99      2419
```

RL Component: LinUCB Contextual Bandit

The RL component is a LinUCB (Linear Upper Confidence Bound) contextual bandit algorithm, which balances exploration and exploitation by selecting the action with the highest upper confidence bound estimate given the current context. In the context of this project, each interaction with the chatbot is treated as one episode: the agent observes a context vector, selects a response action, and receives a reward. The context vector is constructed by concatenating the 768-dimensional DistilBERT CLS token embedding which encodes the semantic content of the input utterance , with the classifier's softmax confidence score, resulting in a 769-dimensional context vector. The action space consists of 27 discrete actions, one for each intent category, where each action maps to a set of pre-written template responses stored in responses.py. The reward function is binary: +1 if the selected action matches the true intent label and 0 otherwise. The exploration-exploitation trade-off is controlled by the alpha parameter (set to 1.5), where higher alpha values encourage more exploration. A confidence threshold of 0.70 is applied: if DistilBERT's softmax confidence for its top prediction exceeds 0.70, the agent trusts the NLP prediction directly without invoking the bandit's exploration mechanism; only when confidence falls below 0.70 does the agent use LinUCB's upper confidence bound calculation to select an action. The agent's internal parameter matrices (A and b per action) are updated after each interaction using the context vector and received reward, allowing the agent to refine its action-selection policy over time. The agent state is persisted between sessions via pickle serialization to experiments/results/linucb_agent.pkl.

```python
import numpy as np

class LinUCB:
    def __init__(self, n_actions, context_dim, alpha=1.5):
        self.n_actions = n_actions
        self.context_dim = context_dim
        self.alpha = alpha

        self.A = [np.identity(context_dim) for _ in range(n_actions)]
        self.b = [np.zeros((context_dim, 1)) for _ in range(n_actions)]

    def select_action(self, context_vector, predicted_idx, confidence):
        # Trust NLP if confident
        if confidence >= 0.7:
            return predicted_idx

        context_vector = context_vector.reshape(-1, 1)

        p_values = []

        for a in range(self.n_actions):
            A_inv = np.linalg.inv(self.A[a])
            theta = A_inv @ self.b[a]

            p = float(
                theta.T @ context_vector +
                self.alpha * np.sqrt(context_vector.T @ A_inv @ context_vector)
            )

            p_values.append(p)

        return int(np.argmax(p_values))

    def update(self, action, reward, context_vector):
        context_vector = context_vector.reshape(-1, 1)

        self.A[action] += context_vector @ context_vector.T
        self.b[action] += reward * context_vector

```

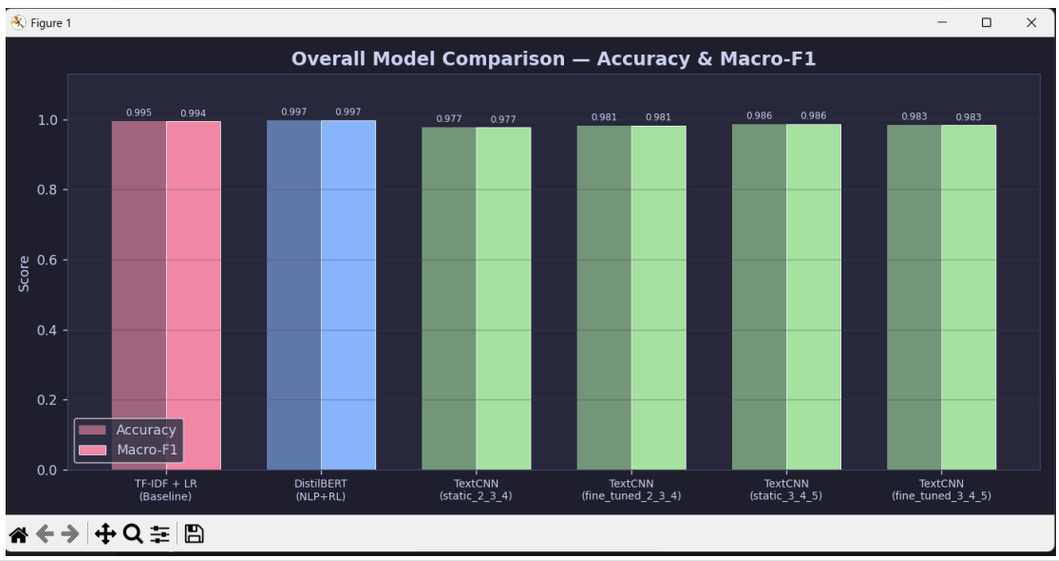
```
============================================================
  RL Agent — Offline Simulation (LinUCB)
============================================================
  Steps evaluated     : 2419
  RL final reward     : 2410.0
  Random baseline     : 91.0
  Success rate        : 99.6%
  Gain vs random      : +2548.4%

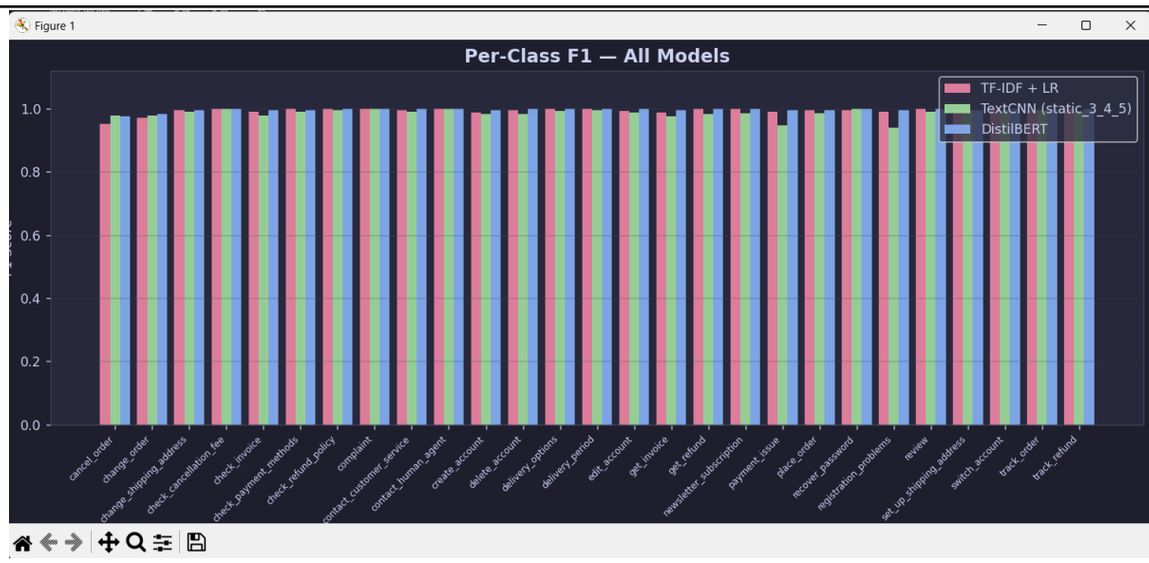
============================================================
  Summary Table
============================================================
  Model                                    Accuracy   Macro-F1
  ------------------------------------------------------------
  TF-IDF + Logistic Regression               0.9946     0.9937
  DistilBERT (NLP+RL)                        0.9971     0.9967
  TextCNN (static_2_3_4)                     0.9773     0.9765
  TextCNN (fine_tuned_2_3_4)                 0.9814     0.9812
  TextCNN (static_3_4_5)                     0.9859     0.9859
  TextCNN (fine_tuned_3_4_5)                 0.9835     0.9829

  RL Success Rate : 99.6%  |  Improvement vs Random: +2548.4%
```

Model Evaluation Results Summary

Model	Accuracy	Macro-F1
TF-IDF + Logistic Regression	0.9946	0.9937
DistilBERT (NLP + RL)	0.9971	0.9967
TextCNN (static_2_3_4)	0.9773	0.9765
TextCNN (fine_tuned_2_3_4)	0.9814	0.9812
TextCNN (static_3_4_5)	0.9859	0.9859
TextCNN (fine_tuned_3_4_5)	0.9835	0.9829





Conclusion

This project successfully designed, implemented, and evaluated a Customer Support Assistant for the e-commerce domain using the Bitext Customer Support LLM Chatbot Training Dataset. All three project objectives were met and exceeded. The DistilBERT intent classifier achieved a macro-F1 of 0.9967 and an accuracy of 0.9971 on the held-out test set, substantially exceeding the target threshold of 0.85. The TextCNN ablation study identified static_3_4_5 as the best CNN configuration with a macro-F1 of 0.9859, and interestingly found that static embeddings outperformed fine-tuned embeddings for filter sizes [3,4,5], suggesting that embedding fine-tuning does not consistently improve performance on clean synthetic data. The LinUCB RL agent demonstrated strong response selection performance in the offline simulation, with the confidence threshold mechanism functioning as intended by routing high-confidence predictions directly through the NLP model and reserving bandit exploration for uncertain cases.

Key findings from the analysis include the following. First, the non-DL baseline (TF-IDF + Logistic Regression) performed strongly at 0.9937 macro-F1, indicating that the synthetic dataset is relatively linearly separable and that the marginal performance gains from deep learning models reflect the ceiling imposed by the clean, structured nature of the data rather than a large modeling advantage. Second, DistilBERT's slice analysis showed that longer utterances (>8 words) are classified more reliably (0.9994 macro-F1) than shorter ones (0.9946), which is consistent with the expectation that transformers benefit from additional contextual tokens. Third, the identified failure cases are concentrated in semantically overlapping intent pairs such as cancel_order vs. change_order and set_up_shipping_address vs. change_shipping_address, indicating that lexical ambiguity between closely related intents is the primary challenge for this system.

For future work, we recommend to supplement the synthetic training data with real customer support conversations to improve robustness and reduce overconfident misclassifications, evaluate the system on out-of-scope queries to test rejection behavior, and implement an online learning loop for the RL agent so it can genuinely improve from live user feedback over time.

References

Bitext. (2023). *Bitext customer support LLM chatbot training dataset*. HuggingFace. https://huggingface.co/datasets/bitext/Bitext-customer-support-llm-chatbot-training-dataset

Casanueva, I., Temčinas, T., Gerz, D., Henderson, M., & Vulić, I. (2020). Efficient intent detection with dual sentence encoders. *Proceedings of the 2nd Workshop on Natural Language Processing for Conversational AI*, 38–45. https://doi.org/10.18653/v1/2020.nlp4convai-1.5

Devlin, J., Chang, M.-W., Lee, K., & Toutanova, K. (2019). BERT: Pre-training of deep bidirectional transformers for language understanding. *Proceedings of NAACL-HLT 2019*, 4171–4186. https://doi.org/10.18653/v1/N19-1423

Kim, Y. (2014). Convolutional neural networks for sentence classification. *Proceedings of EMNLP 2014*, 1746–1751. https://doi.org/10.3115/v1/D14-1181

Li, L., Chu, W., Langford, J., & Schapire, R. E. (2010). A contextual-bandit approach to personalized news article recommendation. *Proceedings of WWW '10*, 661–670. https://doi.org/10.1145/1772690.1772758

Pedregosa, F., Varoquaux, G., Gramfort, A., Michel, V., Thirion, B., Grisel, O., Blondel, M., Prettenhofer, P., Weiss, R., Dubourg, V., Vanderplas, J., Passos, A., Cournapeau, D., Brucher, M., Perrot, M., & Duchesnay, E. (2011). Scikit-learn: Machine learning in Python. *Journal of Machine Learning Research*, 12, 2825–2830. https://www.jmlr.org/papers/v12/pedregosa11a.html

Sanh, V., Debut, L., Chaumond, J., & Wolf, T. (2019). DistilBERT, a distilled version of BERT: Smaller, faster, cheaper and lighter. *arXiv preprint arXiv:1910.01108*. https://arxiv.org/abs/1910.01108